One network for renders and video: vendor-neutral neural temporal super-resolution and low-latency frame generation trained on commodity cloud GPUs

Anonymous Author

*Anonymous institution.

Corresponding author(s). E-mail(s): anonymous@example.org;

Abstract

Neural upscaling and frame generation now set image quality in many games, but the strongest commercial systems need one vendor's recent GPUs, and research systems need engine data that ordinary video lacks. We present NeuralSS, an open system that serves both cases with one model per task. Its temporal super-resolution network is a recurrent, re-parameterisable convolutional network with a built-in motion estimator that runs only when the engine supplies no motion vectors. Its frame-generation network fine-tunes the open RIFE 4.26 interpolation network, adds engine hints through zero-initialised channels, extrapolates with a late-latched camera prior and blends user-interface pixels towards the newest real frame. Both are trained in 10.13 and 10.07 hours on two NVIDIA T4 GPUs from public datasets, after copy detection that left no duplicate across splits, and both export to ONNX. On five test suites the 3x super-resolution student (16.1 GMAC per 1080p frame) beats bicubic upscaling on ordinary video by 1.19 to 2.19 dB PSNR-Y and an analytic temporal anti-aliasing upscaler by 2.79 to 4.91 dB with engine data, with lower temporal-change error. The largest frame-generation tier beats unchanged RIFE 4.26 on the Middlebury interpolation set (34.23 against 33.34 dB) and frame repetition in extrapolation by 1.86 to 5.57 dB, but the 12.7-GMAC tier is still 1.78 dB below RIFE on Middlebury, the cut detector false-alarms on multi-frame gaps, and eager PyTorch on a T4 needs 27.7 and 48.1 ms per 1080p frame, above real-time budgets. We report these limits with the gains.

Keywords: Temporal super-resolution, Frame generation, Video frame interpolation, Real-time rendering, Latency, Efficient neural networks

1 Introduction

Games increasingly render fewer pixels than they display. A temporal upscaler reconstructs each output frame from a jittered low-resolution render and the history of earlier frames, and a frame generator inserts synthetic frames between rendered ones. Neural versions of both now ship in vendor SDKs, but the strongest paths need recent hardware of one vendor: DLSS 4 multi-frame generation relies on RTX 50 Series GPUs[1], and AMD's machine-learning upscaling and frame generation require RDNA 4 GPUs, with analytical fallbacks

[1] https://research.nvidia.com/labs/adlr/DLSS4, accessed 8 November 2026.

on older ones[2]. Intel's XeSS super-resolution runs on any GPU with Shader Model 6.4[3]. Research systems are open but assume a full set of engine buffers [1–3], so they cannot serve captured or streamed video, where only colour exists. Video frame interpolation networks such as RIFE [4] work from colour alone but ignore the motion vectors, depth and user-interface (UI) layers that an engine can provide, and they do not extrapolate.

This paper asks whether one compact model per task can serve engine renders with motion vectors and plain video without them, at a cost that fits integrated and entry-level GPUs, when trained with free cloud compute and public data only. Each network has tiers sized in multiply-accumulates (MAC) per 1080p output frame. Frame generation is designed around input latency rather than frame rate, so it supports extrapolation, which holds no rendered frame back. Both networks reduce to convolutions, pixel shuffles and bilinear warps that export to ONNX.

The contributions are:

1. A recurrent temporal super-resolution (TSR) network with a built-in motion estimator, trained on mixed render-like and plain-video degradations so that one model serves both, with tiers from 6.2 to 156.1 GMAC per 1080p frame at 3x (Section 4).
2. A frame-generation (FG) network that fine-tunes RIFE 4.26 and adds engine hints through zero-initialised channels. The fresh IFNet blend reproduces RIFE 4.26, but the fresh head mixes the newest real frame in at UI weight $\sigma(-4) \approx 0.018$. The network adds a learned UI weight, a late-latched camera prior for extrapolation and a scene-cut gate (Section 5).
3. A leakage-audited data pipeline over 12,300 public items and an evaluation protocol that tests every suite in an engine mode and a plain-video mode, with temporal, UI, cut and latency measures (Sections 3 and 6).
4. Measurements on two Kaggle T4 GPUs that include the failures: the small FG tiers stay below RIFE, the plain-video cut gate false-alarms, and eager latency exceeds the tier budgets (Sections 7 and 8).

[2] https://gpuopen.com/learn/amd-fsr-redstone-developers-neural-rendering/, accessed 8 November 2026.

[3] https://github.com/intel/xess, accessed 8 November 2026.

Figure 1 summarises both networks.

2 Related work

Temporal anti-aliasing and upscaling. Temporal anti-aliasing reprojects an accumulated history with motion vectors and rectifies it against the current frame's neighbourhood to suppress ghosting, and temporal upsampling extends the same idea to lower render resolutions [5]. *Neural supersampling for rendering.* NSRR reconstructs high-resolution frames from the colour, depth and dense motion vectors of the current and previous frames [1]. Later work combines denoising and supersampling in a temporally stable network [6], reduces cost on a new gaming dataset [7], fuses high-resolution G-buffers with low-resolution colour [2], demodulates lighting from materials [8] and decouples G-buffer guidance [9]. All assume engine buffers at inference. Our TSR network uses them when present and estimates motion itself when they are absent.

Frame generation for rendering. Interpolation for rendered content can use auxiliary buffers [10], but it holds a rendered frame back. Extrapolation avoids that latency: ExtraNet uses G-buffers of the new frame and in-paints disocclusions [3], ExtraSS joins extrapolation with supersampling [11], space-time supersampling treats both in one network [12], GFFE removes the need for the new frame's G-buffers [13], and Mob-FGSR targets mobile GPUs [14].

Video frame interpolation. RIFE estimates intermediate flows coarse to fine with an IFNet at an arbitrary time step and runs in real time [4]. IFRNet [15], FILM [16] and EMA-VFI [17] trade more computation for accuracy or large motion. Logos, UI and subtitles break interpolation, and a discontinuity map with figure-text mixing copies such regions from the previous frame instead of interpolating them [18]. We build on RIFE 4.26 because its weights are open and it is cheap enough to tier.

Efficient networks and temporal metrics. Structural re-parameterisation trains multi-branch blocks and deploys one 3×3 convolution [19, 20], and sub-pixel convolution upsamples cheaply [21]. Recurrent video super-resolution propagates features across frames [22]. TecoGAN defines the tOF and tLP temporal metrics we report [23].

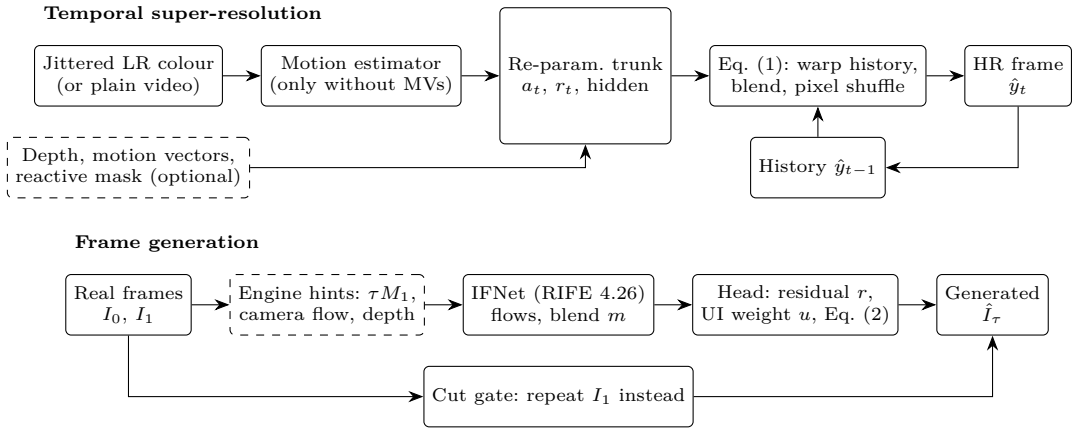


Fig. 1 Overview of the two networks. Dashed boxes are engine inputs that are used when present and dropped at random in training. Both networks reduce to convolutions, pixel shuffles and bilinear warps for ONNX export

3 Data and leakage control

Training and test data come only from public sources: DIV2K [24] and other high-resolution stills, 1080p game frames, REDS [25], Vimeo-90K septuplets and triplets [26], MPI Sintel with exact flow and occlusion [27], TartanAir with exact flow and depth from Unreal Engine environments [28], CARLA [29] renders from GameIR with native 720p and 1440p frames [30], the Vid4 sequences (calendar, city, foliage and walk) evaluated by Liu and Sun [31], and 1080p Vimeo videos. Low-resolution inputs are generated from high-resolution frames, so every sample has ground truth. Online Resource 1 (Section S3.1) lists every source and licence.

Copy detection runs in two stages, because global similarity alone cannot tell the same picture from a similar one. Candidates are the 16 nearest neighbours of each keyframe by self-supervised DINOv2-small embeddings outside its leakage group plus perceptual-hash pairs, and a candidate becomes a duplicate edge when SIFT matches [32] with the ratio test and one RANSAC affine fit [33] cover at least 10 cells of an 8×8 grid, or when its embedding or hash similarity passes a stricter threshold. Thresholds are calibrated on 1,500 synthetic copies against 2,930 random and 998 hard negatives (recall 0.967 at precision 0.999). Leakage groups (a video, a Sintel scene family, a TartanAir environment, a CARLA town) are the unit of the split, and any training item linked to a test item by a duplicate edge is dropped. An audit then compares every validation keyframe with its 5 nearest training keyframes, and every test keyframe with its nearest training and validation keyframes, by cosine similarity and by pHash.

On 12,300 items the split is 8,874 training, 1,087 validation and 1,432 test items, with 756 dropped and 151 filtered for quality. The audit took 2 rounds: the first found duplicates that the search had missed, and the last found none (Online Resource 1, Sections S3.5 and S3.8).

A GPU renderer adds synthetic game-like sequences with exact ground truth. A background and a foreground layer with procedural alpha (blobs, wires one or two texels wide, fences) move on their own affine paths at four speed classes (static, slow, medium, fast) with optional reversal. Particles write no motion vectors and set a reactive mask, as in real engines. Low-resolution frames take one jittered sample per pixel at Halton (2, 3) offsets without mip filtering, so thin geometry aliases, and targets use four samples per pixel. For FG the renderer adds HUD tiles, subtitle changes, menus and scene cuts (Section S3.10).

4 Temporal super-resolution

Inputs. At render resolution $h \times w$ the network receives the jittered colour, depth and normals derived from it, motion vectors to the previous frame, a depth-based disocclusion cue, the reactive mask, exposure and jitter offsets, flags for depth availability and motion quality (1 for exact vectors, 0.5 for RAFT, 0.25 for the built-in estimator, 0 for none), an input-kind flag (render

or video), the previous output warped by the motion vectors and folded to render resolution by space-to-depth, and a warped recurrent hidden state.

Motion without engine data. Every tier contains a three-level IFNet motion estimator [4] that computes motion vectors from the previous and current colour frames whenever a sample has none. Its output enters the trunk with motion quality 0.25. It is supervised by exact or RAFT (small model) [34] motion, and during the first quarter of training samples without motion vectors warp the history with the label motion at a falling probability (teacher forcing), so that the trunk does not learn to distrust history while the estimator is poor. With engine motion vectors the estimator is skipped.

Trunk and output. The trunk is a chain of reparameterisable blocks (a 3×3, a 1×1, an identity and fixed Sobel and Laplacian filters with learned projections) folded into one 3×3 kernel [20], with an optional half-resolution U-Net level. In each block the 1×1 and edge-projection branches start with zero weights (their biases keep the default initialisation), which kept activations bounded in FP16. The network predicts a per-pixel history weight a_t and a residual r_t at render resolution, unfolds them by pixel shuffle [21], and forms

$$\hat{y}_t = a_t \odot \mathcal{W}(\hat{y}_{t-1}, \mathbf{m}_t) + (1 - a_t) \odot \mathcal{U}(x_t) + r_t, \quad (1)$$

where $\mathcal{W}$ warps by the upsampled motion $\mathbf{m}_t$ and $\mathcal{U}$ is a jitter-aware bilinear upsampling of the colour x_t. A refinement at output resolution follows in all tiers but the smallest, whose last layer starts at zero. Equation (1) is a learned form of temporal accumulation with rectification: a_t should approach 0 at disocclusions and particles and 1 on static content. After a reset $a_t = 0$.

Training. A share 0.4 of each batch is real video and the rest is synthetic. A share 0.3 of synthetic and 0.6 of real samples become plain video: anti-aliased bicubic, bilinear or area downscaling, blur, noise and 8-bit quantisation, without jitter, depth or reactive mask. Of these, a share 0.15 keep block-averaged motion vectors of quality at most 0.5, and the others have none, so the estimator runs. Every sample loses its motion vectors with probability 0.25 and its depth with probability 0.3. Six-frame unrolls on 192×192 output patches are the default, and every 4-th step is a 16-frame synthetic unroll at 96×96 so that static scenes keep converging. The per-frame loss is a Charbonnier penalty plus 0.1 times a gradient loss, 0.05 times an L1 loss on Fourier magnitudes, 0.05 times LPIPS [35] on the last frame, and 0.5 times a temporal-change loss over pixels visible in both frames, plus 0.1 times the motion loss. Training uses AdamW [36], FP16 autocast with loss scaling [37], a cosine schedule over a wall-clock budget, and an exponential moving average of the weights. Distillation from the teacher gave no measurable gain in earlier runs and is off (weight 0). Online Resource 1 (Section S4) gives every input, equation and hyperparameter.

Table 1 Network tiers: fused parameters, multiply-accumulates per 1080p output frame (GMAC, warps excluded) and measured T4 latency (fused FP16, eager PyTorch, cuDNN autotuning). TSR figures are for 3x and include the motion estimator, which runs only without engine motion vectors (latency with engine vectors in brackets).

Task	Tier	Params	GMAC	T4 ms
TSR	`igpu`	138K	6.2	16.3 (10.0)
	`low`	328K	16.1	27.7 (21.0)
	`mid`	653K	28.1	–
	`high`	1.41M	69.0	–
	teacher	3.40M	156.1	58.7 (50.1)
FG	`igpu`	454K	9.4	46.8
	`low`	953K	12.7	48.1
	`mid`	2.29M	44.3	106.6
	`high`	5.71M	76.7	120.4

Table 1 lists the tiers. The teacher is the same architecture at 64 channels and serves as the quality ceiling.

5 Frame generation

Motion from colour. Given real frames I_0 and I_1 and a target time τ ($0 < \tau < 1$ interpolates, $\tau = 1.5$ extrapolates), an IFNet with RIFE 4.26's design estimates the flows $F_{\tau\to0}$ and $F_{\tau\to1}$, a blend logit and features over five blocks from 1/16 to full resolution. The `high` tier is RIFE 4.26's network and loads its weights, smaller tiers are narrower and inherit leading channel slices. Each tier estimates flow on at most 576 (`igpu`, `low`) or 1152 (`mid`, `high`) image lines, halving taller frames, so training crops always use full-resolution flow.

Engine hints. When an engine supplies the motion vector M_1 of I_1 (displacement to I_0), linear-motion priors τM_1 and $(\tau-1)M_1$, a camera-only flow sampled from the latest camera input, the depth of I_1 and availability flags enter every flow block as extra channels whose weights start at zero. The IFNet blend of a fresh `high` model reproduces RIFE 4.26 (maximum difference about 3×10^{-6} in the CPU model test of the code, not in a run), but the fresh head mixes the newest real frame in at UI weight $\sigma(-4) \approx 0.018$, so the output is not RIFE's frame exactly before fine-tuning. Fine-tuning learns how far to trust each hint, and training drops the hints at random so that one model serves engines and video.

Output and UI. A half-resolution head on the warped frames predicts a residual r and a UI logit, whose sigmoid u is a soft UI weight, and

$$\hat{I}_\tau = u \odot I_1 + (1-u) \odot \big(m \odot \mathcal{W}(I_0, F_{\tau\to 0}) + (1-m) \odot \mathcal{W}(I_1, F_{\tau\to 1}) + r\big), \qquad (2)$$

where m is the blend weight. A pixel with a high UI weight is pulled towards the newest real frame rather than interpolated, so HUD and subtitle text follow the newest frame. If the engine provides HUD-less frames and a UI layer, the layer is composited on every displayed frame instead. The feature encoder depends on one frame only, so a pipeline could compute it once per real frame. No caller does this yet, and the exported graph encodes both frames.

Cuts. A cut gate repeats the newest real frame instead of generating. With motion vectors its score s is the half-L1 distance of 32-bin luma histograms plus twice the mean motion-compensated difference $\bar{e}$ between I_1 and the warp of I_0 by the motion vector of I_1, so $s = d_{\text{hist}} + 2\bar{e}$. Without motion vectors the score is the histogram distance alone. Each score has a threshold calibrated for at most 1 percent false alarms on continuous synthetic pairs.

Camera prior. On synthetic data the prior is the exact background trajectory at time τ. Real video has none, so the prior is zero there.

Training. Synthetic samples provide exact intermediate flows, disocclusions, HUD tiles with subtitle changes, menus, cuts and camera reversals. Real samples are four-frame quads with RAFT flows, a share 0.5 of them without motion vectors, and a share 0.25 of real interpolation samples are 3x multi-frame generation (targets at $\tau = 1/3$ and $2/3$). The loss is a Charbonnier penalty plus 0.5 times a census loss [38], 0.1 times LPIPS, 0.01 times a flow L1 loss (a quarter of that weight on RAFT labels), 0.2 times a binary cross-entropy on the UI mask with positive weight 4, and 0.5 times distillation. Distillation matches the output and the flows (weighted 0.01) of the larger model. The `high` tier is fine-tuned from RIFE 4.26 first (learning rate 0.0001) and then distils into the smaller tiers (learning rate 0.0004). Online Resource 1 (Section S6) gives every equation, tier and hyperparameter.

6 Experimental setup

Hardware and budgets. Each notebook ran once on Kaggle with two NVIDIA T4 GPUs (14.56 GB each), 4 CPU cores and PyTorch 2.11.0+cu130 with CUDA 13.0, under a 10.8-hour run budget within Kaggle's 12-hour session. TSR trained the teacher for 170.5 minutes, the `low` 3x student for 190.3 and the `igpu` 3x and `low` 2x students in parallel for 110.3 minutes (10.13 hours in total). FG trained `high` for 149.2 minutes, `low` for 198.5 and `igpu` and `mid` in parallel for 119.3 minutes (10.07 hours). Ablations ran for 30 minutes each, in pairs. Checkpoints were chosen by validation PSNR averaged over inputs with and without engine data (Online Resource 1, Sections S9.2 and S9.3).

Test suites and modes. TSR is tested at 3x and 2x on synthetic scenes from held-out textures, held-out TartanAir environments, held-out Sintel scene families, Vid4 and Vimeo-1080p test clips, plus native GameIR renders (CARLA-based) at 2x. Every suite runs in two modes: engine mode (a jittered render with exact motion vectors where they exist, otherwise RAFT motion) and plain mode (a bicubic downscale with no jitter, depth or motion vectors). FG is tested on Vimeo-90K triplets, TartanAir, Sintel, Vid4, Vimeo-1080p clips at 544 lines and at native 1072 lines, synthetic suites for UI, menus, cuts and camera reversal, and, after the run, the 12 Middlebury sequences with ground-truth middle frames [39]. FG modes are interpolation and extrapolation ($\tau = 1.5$), each with and without engine hints, and 3x multi-frame generation, which was

Table 2 TSR at 3x, PSNR-Y (dB) per suite. Plain mode: bicubic downscale, no jitter or engine data. Engine mode: jittered render with motion vectors (exact for synthetic, TartanAir and Sintel, RAFT for video) and depth where available. Bold marks the best deployable tier per row (teacher excluded)

	Plain video				Engine data			
Suite	Bicubic	`igpu`	`low`	Teacher	TAA	`igpu`	`low`	Teacher
Synthetic game	31.23	32.65	**33.42**	34.17	29.35	32.72	**34.27**	35.55
TartanAir	32.34	33.55	**34.03**	34.34	30.64	33.00	**33.43**	33.81
Sintel	33.38	34.23	**34.64**	34.99	31.07	33.70	**34.48**	35.08
Vid4	25.24	26.52	**27.00**	27.48	23.96	26.75	**27.32**	28.20
Vimeo-1080p	35.77	36.50	**36.96**	37.33	33.43	36.88	**37.77**	38.57

tested on plain input only (Online Resource 1, Sections S4.18 and S6.18).

Baselines and metrics. TSR baselines are jitter-aware bilinear and bicubic upsampling and an analytic TAA upscaler (history reprojection, a 3×3 YCoCg neighbourhood clamp and a 10 percent blend of the new frame). FG baselines are frame repetition, frame averaging, motion-vector reprojection and RIFE 4.26 unchanged. For TSR we report PSNR and SSIM [40] on luma (PSNR-Y), LPIPS, FLIP [41] on the last frame of the static-convergence test, a temporal-change error (the loss above as a metric) and tOF [23] along exact or RAFT motion. For FG we report PSNR and SSIM on RGB, LPIPS, UI PSNR, a UI ghosting rate (share of UI pixels off by more than 10 percent), the gated share, and tOF on interpolated sequences along the displayed sequence. FLIP and the temporal-change error are not computed for FG (Online Resource 1, Section S2.9). A pacing model simulates a GPU-bound loop on a variable-refresh display and reports displayed frame rate, game latency (input to a real frame that reflects it) and camera latency (input to any frame whose camera reflects it).

Gated samples. The test clips contain no cuts, so every real sample that the gate replaces by a repeated frame is a false alarm. FG tables therefore report samples the gate left alone ("not gated"). Text comparisons of engine and plain inputs use all test samples.

Use of AI tools. A large language model assistant (Anthropic Claude) was used to help write the code, analyse the result files and draft this manuscript. Every number in the paper is taken from the result files of the runs, which are public with the code.

Table 3 TSR ablations at 3x: PSNR-Y difference (dB) to a reference trained with the same ablation budget. P plain mode, E engine mode

Removed	Synthetic	TartanAir	Vid4
History, E	−1.94	−0.71	−2.04
G-buffers, E	−1.75	−0.72	−1.06
Video training, P	−1.58	−1.18	−1.26
Video training, E	+0.45	+0.17	+0.29
Motion net, P	−0.16	−0.21	−0.46
16-frame unrolls, P	−0.56	−0.23	−0.44
16-frame unrolls, E	−0.18	−0.02	−0.08

7 Results

7.1 Temporal super-resolution

Table 2 gives the 3x results (Online Resource 1, Section S5.3) and Table 3 the ablations. On plain video the `low` student beats bicubic upscaling on every suite, by 1.19 to 2.19 dB PSNR-Y. Its LPIPS ranges from 0.075 to 0.147 against 0.205 to 0.400 for bicubic. With engine data it beats the analytic TAA upscaler by 2.79 to 4.91 dB, with lower temporal-change error on all five suites. In plain mode its temporal-change error is lower than bicubic's on the synthetic, TartanAir and Vid4 suites and higher on Sintel (0.0090 against 0.0081) and Vimeo-1080p (0.0060 against 0.0059). The teacher, at 9.68 times the cost of the student, adds 0.38 to 1.29 dB in engine mode and 0.31 to 0.74 dB in plain mode. The `igpu` student, at 6.2 GMAC, keeps most of the gain. At 2x the student leads bicubic by 1.57 to 2.67 dB in plain mode, and on native GameIR renders it scores 32.60 dB against 32.33 dB for bicubic and 30.83 dB for TAA.

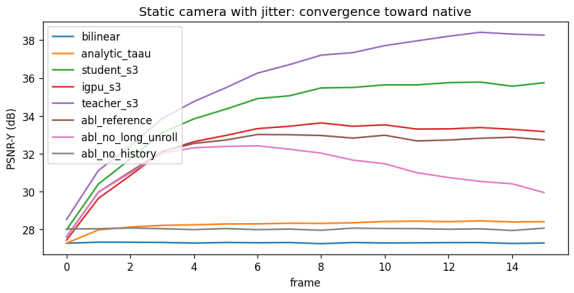


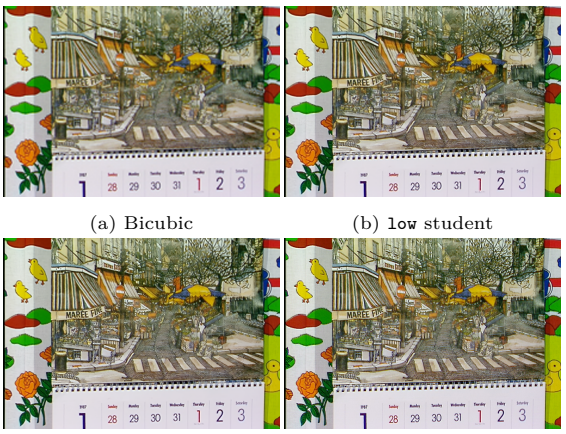
Fig. 2 Static camera with sub-pixel jitter on the synthetic suite: PSNR-Y against frame index for the baselines, the trained tiers and three ablations

Static convergence. With a static camera and sub-pixel jitter a temporal upscaler should approach the native image. At the last frame of the static sequence the student reaches 35.75 dB (FLIP 0.025), the teacher 38.26 dB, TAA 28.42 dB and the single-frame ablation 28.08 dB (Fig. 2). Without the 16-frame unrolls the model peaks early and then falls to 29.96 dB. In plain mode the student's lead over bicubic on static scenes is far smaller (Online Resource 1, Section S5.7).

Ablations. Table 3 isolates the components with ablation runs of the `low` recipe. History and G-buffers are the largest engine-mode effects (0.71 to 2.04 dB). Plain-video training is worth 1.18 to 1.58 dB on plain video and costs nothing measurable on render input, where the model without it is 0.17 to 0.45 dB better. The motion estimator adds 0.16 to 0.46 dB on plain video. An earlier repeat with shorter budgets gave test means within a few tenths of a dB of these, so effects that size are noise. Figure 3 shows Vid4 in plain mode.

7.2 Frame generation

Interpolation. Table 4 compares the tiers with RIFE 4.26 on plain video (Online Resource 1, Sections S7.2.2 and S7.2.5). The fine-tuned `high` tier beats RIFE on Middlebury (+0.89 dB) and on four of the six video suites, by 0.21 to 0.84 dB. It ties on TartanAir (−0.06 dB) and loses 0.50 dB on the 8 Sintel samples. The `mid` tier (44.3 GMAC) is within 0.5 dB of RIFE on every suite except TartanAir and Sintel. The `low` tier beats frame averaging everywhere but trails RIFE by 1.49 to 4.95 dB. Engine hints change the interpolation PSNR of `high` by −0.23 dB on Sintel and +1.42 dB on TartanAir (exact motion). Against RIFE 4.26 without hints, the hinted `high` ranges from



(a) Bicubic (b) `low` student

(c) Teacher (d) Ground truth

Fig. 3 Vid4 calendar at 3x in plain mode (bicubic downscale, no motion vectors). The student sharpens the sign lettering, awning stripes and calendar digits that bicubic upscaling blurs, but adds a fine grid texture on the cobblestones, and the teacher recovers more detail

−0.63 dB on Sintel to +1.36 dB on TartanAir. In 3x multi-frame generation `high` matches or beats RIFE on most suites, including Vimeo-1080p (32.03 against 31.99 dB) and native 1080p (30.74 against 30.45 dB).

Extrapolation. Table 5 shows the mode that adds no held frame (Online Resource 1, Section S7.3). RIFE is not built for $\tau > 1$: it is below frame repetition on Vimeo-1080p and native 1080p, and at most 0.54 dB above it on the others. On plain video `high` beats repetition by 1.86 to 5.57 dB, and `low` by 0.15 to 2.91 dB. With engine motion vectors, motion-vector reprojection remains better on TartanAir (1.79 dB, exact motion).

UI, camera reversal and cuts. On synthetic HUD and subtitle scenes (interpolation with engine hints) the UI ghosting rate is 0.047 for `low` and 0.049 for `high`, against 0.228 for RIFE and 0.302 for reprojection, and 0.010 when the engine supplies a UI layer (Fig. 4). Removing UI training raises it to 0.362. On rapid camera reversals in extrapolation the late-latched camera prior raises `low` from 25.30 to 28.01 dB and `high` from 26.30 to 27.07 dB, while repetition scores 25.91 dB (Online Resource 1, Section S7.6). The gate detected all 48 synthetic cuts in both input modes. With motion vectors it false-alarmed on 0.08 of the Vimeo-1080p samples and 0.20 of the Sintel samples, and on none of the TartanAir, Vid4 and native 1080p samples. Without them it



(a) Target (b) Average (c) RIFE 4.26 (d) `low` (e) `low` + UI layer (f) Predicted UI mask

Fig. 4 Interpolation of a synthetic scene with HUD bars, a crosshair and a changing subtitle. Averaging and RIFE blend the old and new subtitle text, the `low` tier pulls the UI region towards the newest real frame using its predicted UI mask (f)

Table 4 FG interpolation on plain video (no engine hints), PSNR (dB). Video suites use samples the cut gate left alone (count in brackets). Middlebury was measured after the run on CPU with the downloaded weights. Bold marks the best method per row

Suite	Average	RIFE 4.26	`igpu`	`low`	`mid`	`high`
Middlebury (12)	28.02	33.34	30.30	31.56	33.83	**34.23**
Vimeo-90K triplet (45)	26.36	34.87	29.69	32.94	34.38	**35.33**
Vimeo-1080p, 544 lines (9)	32.47	36.93	32.89	34.93	36.52	**37.14**
Vimeo-1080p, native 1072 lines (5)	29.37	33.00	29.58	31.21	33.01	**33.24**
Vid4 (4)	24.09	28.21	26.23	26.72	28.66	**29.05**
TartanAir (9)	21.29	**30.18**	22.08	25.23	28.64	30.11
Sintel (8)	23.60	**26.88**	23.75	24.48	25.97	26.38

Table 5 FG extrapolation ($\tau = 1.5$), PSNR (dB), samples the cut gate left alone. Plain: no engine hints. Engine: with motion vectors, depth and camera prior, against motion-vector reprojection (MV)

Plain	Repeat	RIFE	`low`	`high`
Vimeo-1080p	30.15	27.27	31.31	**33.38**
Native 1080p	28.79	27.01	28.94	**30.65**
Vid4	21.41	21.95	23.93	**25.67**
TartanAir	19.37	19.51	22.28	**24.94**
Sintel	21.90	21.97	22.94	**24.02**
Engine	**MV**		**`low`**	**`high`**
Vimeo-1080p	32.03		32.11	**32.82**
Native 1080p	29.86		29.86	**30.63**
Vid4	25.89		24.99	**25.92**
TartanAir	**26.91**		24.77	25.12
Sintel	**24.20**		24.08	24.14
Synthetic	32.88		33.56	**34.38**

false-alarmed on 0.00 to 0.25 of two-frame real samples, on 0.30 to 0.50 of the 3x multi-frame samples in four suites (none on Vid4), whose real frames are three apart, and on 0.09 of continuous synthetic pairs, against a calibration target of 0.01. *Ablations.* Ablation runs of the `low` recipe show that the RIFE weights matter most on plain video: without inheritance Vimeo-90K interpolation drops from 28.01 to 25.47 dB (Online Resource 1, Section S7.10). Without engine hints in training, extrapolation with exact motion on TartanAir falls from 24.24 to 19.04 dB, but plain-video interpolation rises from 28.01 to 29.19 dB. A model trained only for extrapolation fails at interpolation (21.29 against 32.73 dB with engine hints).

7.3 Real-time cost

Table 1 gives the measured T4 times (Online Resource 1, Section S7.11). They are far above the arithmetic cost: the 12.7-GMAC FG `low` tier takes 48.1 ms and the 6.2-GMAC TSR `igpu` tier 16.3 ms. The `mid` and `high` FG tiers are faster at 1440p (85.7 and 91.2 ms) than at 1080p (106.6 and 120.4 ms) because their flow is halved above 1152 lines. With 48.1 ms per generated frame, the pacing model at a 30 fps render rate drops the displayed rate to 24.56 fps in every FG mode. Interpolation raises game latency from 38.3 to 127.1 ms, while extrapolation keeps it at 38.3 ms and late latching brings camera latency to 43.7 ms. The fused networks export to ONNX with a maximum ONNX Runtime difference of 0.00002792 (TSR)

and 0.00000754 (FG). The graphs are static: the FG graph for a 128×128 input and the TSR graph for a 36×64 render, so another size needs a new export.

8 Discussion

What one model per task achieves. A single TSR network handles jittered renders with engine data and plain video without it. Training on both degradations is what makes plain video work, and it costs the render path nothing measurable (Table 3). Longer budgets did not help in the earlier repeat, so the next TSR gains need more exact-motion data rather than more time. For FG, fine-tuning RIFE 4.26 with zero-initialised hints keeps its quality and adds extrapolation, UI handling and a camera prior.

Where it falls short. The tiers that fit integrated and entry GPUs do not reach RIFE 4.26's quality in FG. The `low` tier stays 1.78 dB below RIFE on Middlebury, with half of RIFE's channel widths and 0.17 of its cost, so capacity now limits it. Extrapolation with exact motion vectors does not beat plain reprojection on TartanAir. The histogram cut gate is not deployable. Most importantly for real-time use, every measured tier exceeds a 16.7 ms frame budget in eager PyTorch on a T4 except the TSR `igpu` tier, which takes 16.3 ms (10.0 ms with engine motion vectors), nearly the whole budget. The TSR `low` tier's 27.7 ms is 3.43 times its arithmetic estimate for a 4-TFLOPS integrated GPU, which points to per-kernel overhead and memory traffic. We infer this from the timings and have not profiled it.

Threats to validity. Each configuration ran once, and only the TSR configuration was repeated with other budgets (Online Resource 1, Section S8.3). Several ungated FG means rest on few samples (4 Vid4 and 5 native 1080p samples). Vimeo-90K makes up 0.91 of the real TSR training windows, and only the Sintel and TartanAir windows have exact motion. Low-resolution inputs are simulated from finished frames, so level-of-detail, shading-rate and lighting effects of real renderers are missing. The plain-video protocol uses clean bicubic downscaling, while real video adds codec artefacts. We did not compare with commercial upscalers or frame generators, which needs engine captures, and latency was measured on a data-centre GPU, not on target hardware.

9 Conclusion

NeuralSS shows that one compact recurrent network can upscale both engine renders and plain video, beating bicubic and analytic TAA upscaling on five suites, and that fine-tuning RIFE 4.26 with zero-initialised engine hints yields a frame generator that beats RIFE at its own width while adding extrapolation, UI handling and a camera prior. The results also mark what remains: the `low` and `igpu` frame-generation tiers are 1.49 to 4.95 and 1.98 to 8.10 dB below RIFE in plain interpolation, cut detection without motion vectors false-alarms, and in eager execution only the smallest super-resolution tier fits a 60 fps frame budget, with almost no margin. Next steps are wider half-resolution-flow FG tiers, cut thresholds calibrated on real clips per frame gap, deployment runtimes timed on integrated GPUs, repeated seeds, and engine captures for a comparison with commercial systems.

Statements and Declarations

Supplementary information. Online Resource 1 (supplementary material) explains every component, equation, training objective and evaluation protocol from first principles and presents every table, plot and prediction of the three notebooks with its interpretation. Its references carry the prefix S.

Funding. No funding was received for this work.

Competing interests. The author has no competing interests to declare.

Data availability. All datasets are public: DIV2K, Flickr2K, REDS, Vimeo-90K, Vid4 and MPI Sintel on Kaggle, and TartanAir, the CARLA-based GameIR renders, 1080p game frames and Vimeo-1080p on Hugging Face. The dataset identifiers, the split manifest, every metric file, plot and log of the runs reported here are available from the corresponding author and will be released with the code on acceptance.

Code availability. The code, notebooks and trained weights will be released on acceptance.

Ethics approval and consent. Not applicable.

Author contributions. Withheld for review.

References

[1] Xiao, L., Nouri, S., Chapman, M., Fix, A., Lanman, D., Kaplanyan, A.: Neural supersampling for real-time rendering. ACM Transactions on Graphics **39**(4) (2020) https://doi.org/10.1145/3386569.3392376

[2] Zhong, Z., Zhu, J., Dai, Y., Zheng, C., Chen, G., Huo, Y., Bao, H., Wang, R.: FuseSR: Super resolution for real-time rendering through efficient multi-resolution fusion. In: SIGGRAPH Asia 2023 Conference Papers, pp. 1–10 (2023). https://doi.org/10.1145/3610548.3618209

[3] Guo, J., Fu, X., Lin, L., Ma, H., Guo, Y., Liu, S., Yan, L.-Q.: ExtraNet: Real-time extrapolated rendering for low-latency temporal supersampling. ACM Transactions on Graphics **40**(6), 1–16 (2021) https://doi.org/10.1145/3478513.3480531

[4] Huang, Z., Zhang, T., Heng, W., Shi, B., Zhou, S.: Real-time intermediate flow estimation for video frame interpolation. In: Computer Vision – ECCV 2022, vol. 13674, pp. 624–642 (2022). https://doi.org/10.1007/978-3-031-19781-9_36

[5] Yang, L., Liu, S., Salvi, M.: A survey of temporal antialiasing techniques. Computer Graphics Forum **39**(2), 607–621 (2020) https://doi.org/10.1111/cgf.14018

[6] Thomas, M.M., Liktor, G., Peters, C., Kim, S., Vaidyanathan, K., Forbes, A.G.: Temporally stable real-time joint neural denoising and supersampling. Proceedings of the ACM on Computer Graphics and Interactive Techniques **5**(3), 1–22 (2022) https://doi.org/10.1145/3543870

[7] Mercier, A., Erasmus, R., Savani, Y., Dhingra, M., Porikli, F., Berger, G.: Efficient neural supersampling on a novel gaming dataset. In: 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 296–306 (2023). https://doi.org/10.1109/ICCV51070.2023.00034

[8] Li, J., Chen, Z., Wu, X., Wang, L., Wang, B., Zhang, L.: Neural super-resolution for real-time rendering with radiance demodulation. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4357–4367 (2024). https://doi.org/10.1109/CVPR52733.2024.00417

[9] Zheng, M., Sun, L., Dong, J., Pan, J.: Efficient video super-resolution for real-time rendering with decoupled G-buffer guidance. In: 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 11328–11337 (2025). https://doi.org/10.1109/CVPR52734.2025.01058

[10] Briedis, K.M., Djelouah, A., Meyer, M., McGonigal, I., Gross, M., Schroers, C.: Neural frame interpolation for rendered content. ACM Transactions on Graphics **40**(6), 1–13 (2021) https://doi.org/10.1145/3478513.3480553

[11] Wu, S., Kim, S., Zeng, Z., Vembar, D., Jha, S., Kaplanyan, A., Yan, L.-Q.: ExtraSS: A framework for joint spatial super sampling and frame extrapolation. In: SIGGRAPH Asia 2023 Conference Papers, pp. 1–11 (2023). https://doi.org/10.1145/3610548.3618224

[12] He, R., Zhou, S., Sun, Y., Cheng, R., Tan, W., Yan, B.: Low-latency space-time supersampling for real-time rendering. Proceedings of the AAAI Conference on Artificial Intelligence **38**(3), 2103–2111 (2024) https://doi.org/10.1609/aaai.v38i3.27982

[13] Wu, S., Vembar, D., Sochenov, A., Panneer, S., Kim, S., Kaplanyan, A., Yan, L.-Q.: GFFE: G-buffer free frame extrapolation for low-latency real-time rendering. ACM Transactions on Graphics **43**(6), 1–15 (2024) https://doi.org/10.1145/3687923

[14] Yang, S., Zhu, Q., Zhuge, J., Qiu, Q., Li, C., Yan, Y., Xu, H., Yan, L.-Q., Jin, X.: Mob-FGSR: Frame generation and super resolution for mobile real-time rendering. In: ACM SIGGRAPH 2024 Conference Papers, pp. 1–11 (2024). https://doi.org/10.1145/3641519.3657424

[15] Kong, L., Jiang, B., Luo, D., Chu, W., Huang, X., Tai, Y., Wang, C., Yang, J.: IFRNet: Intermediate feature refine network for efficient frame interpolation. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1959–1968 (2022). https://doi.org/10.1109/CVPR52688.2022.00201

[16] Reda, F., Kontkanen, J., Tabellion, E., Sun, D., Pantofaru, C., Curless, B.: FILM: Frame interpolation for large motion. In: Computer Vision – ECCV 2022, vol. 13667, pp. 250–266 (2022). https://doi.org/10.1007/978-3-031-20071-7_15

[17] Zhang, G., Zhu, Y., Wang, H., Chen, Y., Wu, G., Wang, L.: Extracting motion and appearance via inter-frame attention for efficient video frame interpolation. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5682–5692 (2023). https://doi.org/10.1109/CVPR52729.2023.00550

[18] Lee, S., Lee, H., Shin, C., Son, H., Lee, S.: Exploring discontinuity for video frame interpolation. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9791–9800 (2023). https://doi.org/10.1109/CVPR52729.2023.00944

[19] Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J.: RepVGG: Making VGG-style convnets great again. In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13728–13737 (2021). https://doi.org/10.1109/CVPR46437.2021.01352

[20] Zhang, X., Zeng, H., Zhang, L.: Edge-oriented convolution block for real-time super resolution on mobile devices. In: Proceedings of the 29th ACM International Conference on Multimedia, pp. 4034–4043 (2021). https://doi.org/10.1145/3474085.3475291

[21] Shi, W., Caballero, J., Huszár, F., Totz, J., Aitken, A.P., Bishop, R., Rueckert, D., Wang, Z.: Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1874–1883 (2016). https://doi.org/10.1109/CVPR.2016.207

[22] Chan, K.C.K., Zhou, S., Xu, X., Loy, C.C.: BasicVSR++: Improving video super-resolution with enhanced propagation and alignment. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5962–5971 (2022). https://doi.org/10.1109/CVPR52688.2022.00588

[23] Chu, M., Xie, Y., Mayer, J., Leal-Taixé, L., Thuerey, N.: Learning temporal coherence via self-supervision for GAN-based video generation. ACM Transactions on Graphics **39**(4) (2020) https://doi.org/10.1145/3386569.3392457

[24] Agustsson, E., Timofte, R.: NTIRE 2017 challenge on single image super-resolution: Dataset and study. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1122–1131 (2017). https://doi.org/10.1109/CVPRW.2017.150

[25] Nah, S., Baik, S., Hong, S., Moon, G., Son, S., Timofte, R., Lee, K.M.: NTIRE 2019 challenge on video deblurring and super-resolution: Dataset and study. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1996–2005 (2019). https://doi.org/10.1109/CVPRW.2019.00251

[26] Xue, T., Chen, B., Wu, J., Wei, D., Freeman, W.T.: Video enhancement with task-oriented flow. International Journal of Computer Vision **127**(8), 1106–1125 (2019) https://doi.org/10.1007/s11263-018-01144-2

[27] Butler, D.J., Wulff, J., Stanley, G.B., Black, M.J.: A naturalistic open source movie for optical flow evaluation. In: Computer Vision – ECCV 2012. Lecture Notes in Computer Science, pp. 611–625 (2012). https://doi.org/10.1007/978-3-642-33783-3_44

[28] Wang, W., Zhu, D., Wang, X., Hu, Y., Qiu, Y., Wang, C., Hu, Y., Kapoor, A., Scherer, S.: TartanAir: A dataset to push the limits

of visual SLAM. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 4909–4916 (2020). https://doi.org/10.1109/IROS45743.2020.9341801

[29] Dosovitskiy, A., Ros, G., Codevilla, F., López, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. Proceedings of Machine Learning Research, vol. 78, pp. 1–16 (2017). http://proceedings.mlr.press/v78/dosovitskiy17a.html

[30] Zhou, L., Han, K., Ling, N., Wang, W., Jiang, W.: GameIR: A large-scale synthesized ground-truth dataset for image restoration over gaming content. In: Computer Vision – ACCV 2024 Workshops. Lecture Notes in Computer Science, pp. 179–196 (2025). https://doi.org/10.1007/978-981-96-2644-1_13

[31] Liu, C., Sun, D.: On Bayesian adaptive video super resolution. IEEE Transactions on Pattern Analysis and Machine Intelligence **36**(2), 346–360 (2014) https://doi.org/10.1109/TPAMI.2013.127

[32] Lowe, D.G.: Distinctive image features from scale-invariant keypoints. International Journal of Computer Vision **60**(2), 91–110 (2004) https://doi.org/10.1023/B:VISI.0000029664.99615.94

[33] Fischler, M.A., Bolles, R.C.: Random sample consensus. Communications of the ACM **24**(6), 381–395 (1981) https://doi.org/10.1145/358669.358692

[34] Teed, Z., Deng, J.: RAFT: Recurrent all-pairs field transforms for optical flow. In: Computer Vision – ECCV 2020, vol. 12347, pp. 402–419 (2020). https://doi.org/10.1007/978-3-030-58536-5_24

[35] Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 586–595 (2018). https://doi.org/10.1109/CVPR.2018.00068

[36] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (2019). https://openreview.net/forum?id=Bkg6RiCqY7

[37] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E.K., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., Wu, H.: Mixed precision training. In: International Conference on Learning Representations (2018). https://openreview.net/forum?id=r1gs9JgRZ

[38] Meister, S., Hur, J., Roth, S.: UnFlow: Unsupervised learning of optical flow with a bidirectional census loss. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, pp. 7251–7259 (2018). https://doi.org/10.1609/aaai.v32i1.12276

[39] Baker, S., Scharstein, D., Lewis, J.P., Roth, S., Black, M.J., Szeliski, R.: A database and evaluation methodology for optical flow. International Journal of Computer Vision **92**(1), 1–31 (2011) https://doi.org/10.1007/s11263-010-0390-2

[40] Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: From error visibility to structural similarity. IEEE Transactions on Image Processing **13**(4), 600–612 (2004) https://doi.org/10.1109/TIP.2003.819861

[41] Andersson, P., Nilsson, J., Akenine-Möller, T., Oskarsson, M., Åström, K., Fairchild, M.D.: FLIP: A difference evaluator for alternating images. Proceedings of the ACM on Computer Graphics and Interactive Techniques **3**(2), 1–23 (2020) https://doi.org/10.1145/3406183